第19天 · 把现有流程迁移到 LangGraph

先理解状态、节点与边，再选择是否使用框架。

标准路线：60分钟；忙碌日：20分钟；扩展练习另计。

今天的目标

- 把流程解释成状态、节点与边。

- 检查标准库主线的节点轨迹。

- 按选学说明运行真实LangGraph示例并分清覆盖范围。

开始之前

- 已经完成第 18 课，或能解释其产物。

- 打开实践包 practice 目录；后文 python 可替换为虚拟环境内解释器的完整路径。

一小时安排

- 15分钟 · 理解概念：阅读本课四节讲解，弄清一个关键机制与适用边界。

- 30分钟 · 亲手操作：先跑基线，再做一次单变量实验，保留前后结果。

- 10分钟 · 检查结果：逐项验收，完成两道自测并阅读解析。

- 5分钟 · 留下记录：写出本课发现、未解决问题和下次第一步。

01 · 理解概念

先认出流程，再引入框架

把“接收问题、检索材料、检查证据、输出回答”写成图，有助于看清每个阶段的输入与输出。节点负责局部工作，边决定下一步，状态负责在节点之间传递必要信息。

本课主线仍使用标准库运行同样的业务步骤。先对照 state 与 node_trace 理解流程，再看可选 LangGraph 示例；框架名不能代替对业务逻辑的解释。

状态应该包含完成任务所需的信息

状态中可以包含问题、检索结果、引用和错误状态。不要把所有日志、完整文件与无限聊天历史都堆进去；状态越大，复制、保存和排查问题越困难。

节点更新状态时应有明确合同。检索节点不应顺手完成批准写入；验证节点也不该静默修改来源。职责清楚才能单独测试，也方便以后替换模型或工具。

持久化与恢复需要明确定义

图框架可以提供检查点与持久化机制，但你仍要决定什么时候保存、恢复到哪里，以及重复运行可能产生什么副作用。第12课的恢复状态和第17课的幂等问题在这里依然存在。

本课选学示例的默认用途是展示真实 StateGraph API，不代表已经部署耐久服务或数据库。内存中的流程运行成功，与进程崩溃后仍可恢复，是不同的验收要求。

框架迁移也要保持评价方式

迁移前后使用同一问题与同一材料，比较引用、停止条件和输出结构。如果结果变了，先找状态流转或参数差异，而不是简单归因于框架更聪明。

optional/langgraph_demo.py 是独立示例，使用自己的固定查询，不自动读取 day19 的配置。主线参数实验只作用于 python run.py day 19，避免修改一处配置却期待另一个脚本改变。

02 · 一步一步做

命令在解压后的 practice 文件夹执行。Windows可使用 .\.venv\Scripts\python.exe；macOS/Linux可使用 .venv/bin/python 替代 python。

1. 找到本课的输入与配置

为什么做：先知道程序读什么，后面才能解释输出为何改变。

在 VS Code 中打开 experiments.json，定位 day19。用下面的命令确认当前目录正确，并把各参数的作用写在笔记中。

```
python -c "import json; print(json.load(open('experiments.json', encoding='utf-8'))['day19'])"
```

预期结果：终端显示本课配置；键名与文件中的内容一致。

2. 运行本课基线

为什么做：保持默认设置的一次运行，是后续比较的参照。

执行命令，打开新生成的 outputs/day19.json。先检查 day、mode 和 result，不急着修改代码。

```
python run.py day 19
```

预期结果：结果写入 outputs/day19.json；主线 mode 明确为 offline，代表规则教学流程。

3. 沿输出检查机制

为什么做：不能只看程序有没有报错，要检查它实际做了什么。

格式化阅读输出，重点检查state 与 node_trace。把一个输入参数和一个输出字段用箭头对应起来。

```
python -m json.tool outputs/day19.json
```

预期结果：能够定位state 与 node_trace，并用自己的话解释该结果。

4. 改变一个条件并比较

为什么做：主动制造可解释的变化，才能验证自己理解的机制。

完成下面“改一改”实验。对比基线与新结果，记录哪些字段改变、哪些保持稳定；不要同时改模型、材料和参数。

预期结果：留下一组前后结果和一句解释；失败状态也可以是正确的实验产物。

改一改 · 缩小检索结果，观察状态在节点间的变化

- 先执行 python run.py day 19，将 outputs/day19.json 复制为自己的基线备份。

- 在 VS Code 打开 experiments.json，只修改 day19.top_k 为 1，保持其他键不变并保存。

- 执行 python -m json.tool experiments.json 检查语法，再执行 python run.py day 19。

- 对比两份结果，重点观察state 与 node_trace；解释改变原因后恢复原设置或明确记录保留值。

观察什么：state.top_k反映新的上限；如果问题只命中一条材料，top_k从3降为1时证据数量可能不变。检查真实结果，并用node_trace追踪执行顺序。

机制解释：图结构控制步骤之间的关系，节点内部参数控制具体业务行为，二者应分开比较。

常见问题

修改配置后结果没有变化

原因：改了其他天的字段，或编辑后尚未保存。

处理：核对 experiments.json 中的 day19，保存后重新运行；检查实际执行目录。

把受控失败误当成程序坏了

原因：本课实验主动制造了边界条件。

处理：先对照 expected 和状态字段，区分设计内拒绝与未处理异常；保留错误信息再修复。

03 · 验收

先理解状态、节点与边，再选择是否使用框架。 保留基线、修改结果与自己的解释。

成果文件：outputs/day19.json

☐ 能按 node_trace 还原执行顺序。

☐ 指出每个节点读写的状态字段。

☐ 明确区分主线实验、可选框架示例与未实现持久化。

自测与解析

1. 引入LangGraph后，谁负责定义工具与状态的业务含义?

- A. 框架自动推断全部含义

- B. 项目开发者

- C. 操作系统

- D. 任何网页文本

答案：B。 框架提供运行结构，业务合同仍由开发者设计与验证。

2. 修改day19配置后，独立的可选示例一定变化吗?

- A. 一定

- B. 只有它明确读取此配置才会变化

- C. 因为同一文件夹所以变化

- D. 只要安装依赖就变化

答案：B。 必须理解实际的数据读取路径，不能凭文件位置假定共享配置。

小词典

- State：任务执行过程中明确传递和更新的数据。

- Node：承担某一阶段工作的函数。

- Edge：决定节点之间执行关系的连接。

04 · 我的记录

- 我的流程哪里需要条件分支，哪里只需要固定顺序？

- 引入框架后，哪个行为必须保持不变才能公平比较？

忙碌日：20分钟

- 用5分钟阅读第一个概念，写出本课要解决的问题。

- 用10分钟运行 python run.py day 19，打开 outputs/day19.json，检查一个关键字段。

- 用5分钟完成第一道自测并留一句解释，完整实验与验收留到下次。

保存一份运行结果和自己的解释；精简练习单独记录，不等于整课验收全部完成。

有余力再做：20分钟选学 · 运行独立 LangGraph 示例

- 阅读 practice/optional 中的安装说明，在虚拟环境安装锁定的选学依赖。

- 运行 python optional/langgraph_demo.py，检查节点与最终状态。

- 说明它使用独立固定问题，没有读取 day19 配置；不要把这次结果算作真实模型推理。

参考资料与阅读范围

LangGraph：官方概览：阅读最小StateGraph示例与状态、节点、边的关系。

https://docs.langchain.com/oss/python/langgraph/overview

LangGraph：持久化：选读检查点、线程标识与恢复的基本概念。

https://docs.langchain.com/oss/python/langgraph/persistence

下一步

下一课把研究、质疑与复核拆成可以检查的角色职责。